

Week_4

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

ALGORITHM: Parent Creates Multiple Children and Synchronizes using waitpid()	
1.	START
2.	Declare an array <code>child[NUM_CHILDREN]</code> to store child PIDs and an integer status.
3.	Print a message indicating the parent process has started along with its PID.
4.	For $i \leftarrow 0$ to <code>NUM_CHILDREN - 1</code> do 4.1 Call <code>fork()</code> and store the return value in <code>child[i]</code>. 4.2 If <code>fork() < 0</code> then 4.3 Print error message and terminate the program. 4.4 Else if <code>fork() == 0</code> then // Child process 4.4.1 Print message: "Child $i + 1$ started" along with child PID. 4.4.2 Sleep for $(i + 1) \times 2$ seconds (to create different completion times). 4.4.3 Print message: "Child $i + 1$ completed". 4.4.4 Exit the child process with exit status $(i + 1)$.
5.	End For
6.	Print message: Parent waiting for children using <code>waitpid()</code> .
7.	For $i \leftarrow 0$ to <code>NUM_CHILDREN - 1</code> do
8.	Call <code>waitpid(child[i], &status, 0)</code> and store the return value in <code>pid</code> .
9.	If <code>waitpid() == -1</code> then
10.	Print error message and terminate the program.
11.	Else
12.	If <code>WIFEXITED(status)</code> is true then
13.	Print: Child with PID <code>pid</code> terminated with exit status <code>WEXITSTATUS(status)</code> .
14.	End If
15.	End If
16.	End For
17.	Print message: All child processes completed. Parent exiting. STOP

Sample Output:

Parent Process Started

Parent PID: 4521

Child 1 started. PID = 4522, Parent PID = 4521

Child 2 started. PID = 4523, Parent PID = 4521

Child 3 started. PID = 4524, Parent PID = 4521

All 3 child processes have been created.

Using `wait()`:

Child 1 completed. PID = 4522

wait() detected child PID 4522 finished with exit status 10

Using waitpid():

Child 2 completed. PID = 4523

Child 3 completed. PID = 4524

waitpid() detected child PID 4522 finished with exit status 10

waitpid() detected child PID 4523 finished with exit status 11

waitpid() detected child PID 4524 finished with exit status 12

Parent process completed.

```
alekhya@Alekhyareddy: ~/OS × + v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano task1_wait.c
alekhya@Alekhyareddy:~/OSSP$ gcc task1_wait.c -o task1_wait
alekhya@Alekhyareddy:~/OSSP$ ./task1_wait
Parent Process Started
Parent PID: 409

All 3 child processes have been created.

Using wait():

Child 1 started. PID = 410, Parent PID = 409
Child 2 started. PID = 411, Parent PID = 409
Child 3 started. PID = 412, Parent PID = 409
Child 1 completed. PID = 410
wait() detected child PID 410 finished with exit status 10

Using waitpid():

Child 2 completed. PID = 411
waitpid() detected child PID 411 finished with exit status 11
Child 3 completed. PID = 412
waitpid() detected child PID 412 finished with exit status 12

Parent process completed.
alekhya@Alekhyareddy:~/OSSP$ |
```

```
alekhya@Alekhyareddy: ~/OS × + v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t child[3];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n\n", getpid());

    /* Create 3 child processes */
    for (int i = 0; i < 3; i++)
    {
        child[i] = fork();

        if (child[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (child[i] == 0)
        {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());

            /* Different execution times */
            sleep(i + 1);

            printf("Child %d completed. PID = %d\n",
                i + 1, getpid());

            /* Exit status: 10, 11, 12 */
            exit(10 + i);
        }
    }

    printf("\nAll 3 child processes have been created.\n");
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T E
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J
```

```
alekhya@Alekhyareddy: ~/OS × + v
GNU nano 7.2

printf("\nAll 3 child processes have been created.\n");

/* ----- WAIT() ----- */
printf("\nUsing wait():\n\n");
pid_t finished_pid = wait(&status);
if (WIFEXITED(status))
{
    printf("wait() detected child PID %d finished "
        "with exit status %d\n",
        finished_pid,
        WEXITSTATUS(status));
}

/* ----- WAITPID() ----- */
printf("\nUsing waitpid():\n\n");
for (int i = 0; i < 3; i++)
{
    /* Skip the child already collected by wait() */
    if (child[i] == finished_pid)
        continue;

    pid_t result = waitpid(child[i], &status, 0);

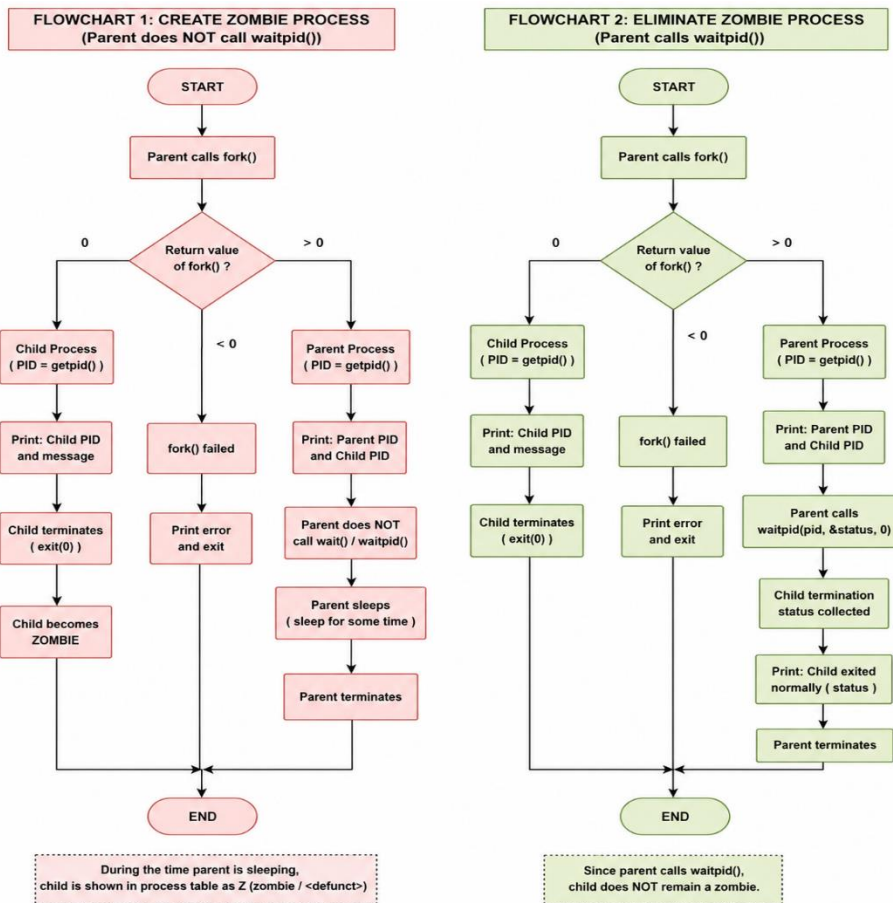
    if (result > 0 && WIFEXITED(status))
    {
        printf("waitpid() detected child PID %d finished "
            "with exit status %d\n",
            result,
            WEXITSTATUS(status));
    }
}

printf("\nParent process completed.\n");

return 0;
}
```

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Flow Chart:



Sample Output:

Parent process: PID = 158

Child process: PID = 159

Child PID = 159

Child process terminating...

Parent sleeping...

Parent terminating...

Ps -el

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
---	---	-----	-----	------	---	-----	----	------	----	-------	-----	------	-----

```

0 S 0 1 0 0 80 0 - 2238 ? ? 00:00:00 init
0 S 0 8 1 0 80 0 - 2238 ? tty1 00:00:00 init
0 S 1000 9 8 0 80 0 - 3590 ? tty1 00:00:00 bash
0 S 0 141 1 0 80 0 - 2238 ? tty2 00:00:00 init
0 S 1000 142 141 0 80 0 - 3527 ? tty2 00:00:00 bash
0 S 1000 158 9 0 80 0 - 2680 ? tty1 00:00:00 week4_T2
0 Z 1000 159 158 0 80 0 - 0 ? tty1 00:00:00 week4_T2
0 R 1000 160 142 0 80 0 - 4086 ? tty2 00:00:00 ps

```

```

alekhya@Alekhyareddy: ~/OS  ×  +  v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano week4_T2.c
alekhya@Alekhyareddy:~/OSSP$ gcc week4_T2.c -o week4_T2
alekhya@Alekhyareddy:~/OSSP$ ./week4_T2
Parent process: PID = 504
Parent sleeping...
Child process: PID = 505
Child PID = 505
Child process terminating...
Parent terminating...
alekhya@Alekhyareddy:~/OSSP$ ps -el
F S  UID      PID      PPID  C  PRI  NI  ADDR  SZ  WCHAN  TTY      TIME  CMD
4 S   0         1         0  0  80   0   -   5393  -   ?      00:00:00 systemd
5 S   0         2         1  0  80   0   -   795  -   hvcc0  00:00:00 init-systemd(Ub
0 S   0         6         2  0  80   0   -   799  -   hvcc0  00:00:00 init
4 S   0         52        1  0  79  -1  - 12609  -   ?      00:00:00 systemd-journal
4 S   0        100        1  0  80   0   -  6472  -   ?      00:00:00 systemd-udevd
4 S  991        114        1  0  80   0   -  5336  -   ?      00:00:00 systemd-resolve
4 S  996        115        1  0  80   0   - 22759  -   ?      00:00:00 systemd-timesyn
4 S   0        167        1  0  80   0   -  1061  -   ?      00:00:00 cron
4 S  101        168        1  0  80   0   -  2399  -   ?      00:00:00 dbus-daemon
4 S   0        176        1  0  80   0   -  4495  -   ?      00:00:00 systemd-logind
4 S   0        178        1  0  80   0   - 439157  -   ?      00:00:00 wsl-pro-service
4 S   0        192        1  0  80   0   -   781  -   tty1   00:00:00 agetty
4 S  102        193        1  0  80   0   - 55629  -   ?      00:00:00 rsyslogd
4 S   0        204        1  0  80   0   - 26759  -   ?      00:00:00 unattended-upgr
4 S   0        271        2  0  80   0   -  1674  -   pts/1  00:00:00 login
4 S 1000        335        1  0  80   0   -  5082  -   ?      00:00:00 systemd
5 S 1000        336        335  0  80   0   -  5292  -   ?      00:00:00 (sd-pam)
4 S 1000        356        271  0  80   0   -  1520  poll_s pts/1  00:00:00 bash
5 S   0        478        2  0  80   0   -   797  -   ?      00:00:00 SessionLeader
5 S   0        479        478  0  80   0   -   801  -   ?      00:00:00 Relay(481)
4 S 1000        481        479  0  80   0   -  1520  do_wai pts/0  00:00:00 bash
5 S   0        509        100  0  80   0   -  6473  -   ?      00:00:00 (udev-worker)
5 S   0        510        100  0  80   0   -  6473  -   ?      00:00:00 (udev-worker)
0 R 1000        511        481  0  80   0   -  2093  -   pts/0  00:00:00 ps
alekhya@Alekhyareddy:~/OSSP$ nano week4_T2.c
alekhya@Alekhyareddy:~/OSSP$ gcc week4_T2.c -o week4_T2
alekhya@Alekhyareddy:~/OSSP$ ./week4_T2
Parent process: PID = 518
Parent waiting for child...
Child process: PID = 519
Child PID = 519
Child process terminating...
Child process collected.
Parent terminating...
alekhya@Alekhyareddy:~/OSSP$ ps -el

```

```

alekhya@Alekhyareddy: ~/OS
5 S      0      510      100 0    80    0 - 6473 -    ?      00:00:00 (udev-worker)
0 R 1000    511      481 0    80    0 - 2093 -    pts/0    00:00:00 ps
alekhya@Alekhyareddy:~/OSSP$ nano week4_T2.c
alekhya@Alekhyareddy:~/OSSP$ gcc week4_T2.c -o week4_T2
alekhya@Alekhyareddy:~/OSSP$ ./week4_T2
Parent process: PID = 518
Parent waiting for child...
Child process: PID = 519
Child PID = 519
Child process terminating...
Child process collected.
Parent terminating...
alekhya@Alekhyareddy:~/OSSP$ ps -el
F S      UID      PID      PPID  C PRI  NI ADDR SZ WCHAN  TTY      TIME CMD
4 S      0         1         0  0  80    0 - 5393 -    ?      00:00:00 systemd
5 S      0         2         1  0  80    0 - 795 -    hvc0    00:00:00 init-systemd(Ub
0 S      0         6         2  0  80    0 - 799 -    hvc0    00:00:00 init
4 S      0         52        1  0  79   -1 - 12609 -    ?      00:00:00 systemd-journal
4 S      0        100        1  0  80    0 - 6472 -    ?      00:00:00 systemd-udev
4 S 991      114        1  0  80    0 - 5336 -    ?      00:00:00 systemd-resolve
4 S 996      115        1  0  80    0 - 22759 -    ?      00:00:00 systemd-timesyn
4 S      0        167        1  0  80    0 - 1061 -    ?      00:00:00 cron
4 S 101      168        1  0  80    0 - 2399 -    ?      00:00:00 dbus-daemon
4 S      0        176        1  0  80    0 - 4495 -    ?      00:00:00 systemd-logind
4 S      0        178        1  0  80    0 - 439157 -    ?      00:00:00 wsl-pro-service
4 S      0        192        1  0  80    0 - 781 -    tty1    00:00:00 agetty
4 S 102      193        1  0  80    0 - 55629 -    ?      00:00:00 rsyslogd
4 S      0        204        1  0  80    0 - 26759 -    ?      00:00:00 unattended-upgr
4 S      0        271        2  0  80    0 - 1674 -    pts/1    00:00:00 login
4 S 1000     335        1  0  80    0 - 5082 -    ?      00:00:00 systemd
5 S 1000     336      335  0  80    0 - 5292 -    ?      00:00:00 (sd-pam)
4 S 1000     356      271  0  80    0 - 1520 poll_s pts/1    00:00:00 bash
5 S      0        478        2  0  80    0 - 797 -    ?      00:00:00 SessionLeader
5 S      0        479      478  0  80    0 - 801 -    ?      00:00:00 Relay(481)
4 S 1000     481      479  0  80    0 - 1520 do_wai pts/0    00:00:00 bash
0 R 1000     522      481  0  80    0 - 2093 -    pts/0    00:00:00 ps
alekhya@Alekhyareddy:~/OSSP$ wait(NULL);
-bash: syntax error near unexpected token `NULL'
alekhya@Alekhyareddy:~/OSSP$

```

```

alekhya@Alekhyareddy: ~/OS
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child PID = %d\n", getpid());
        printf("Child process terminating...\n");

        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child process collected.\n");
        printf("Parent terminating...\n");
    }

    return 0;
}

```